

Project 1: Colorizing the Prokudin-Gorskii Photo Collection

Contents

Background
Context

Overview

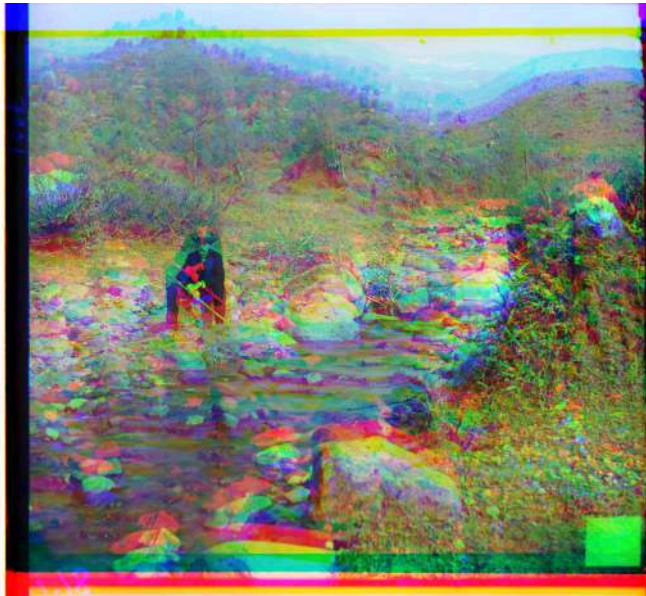
Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



Prokudin-Gorskii's self-portrait before aligning



My auto-aligned glass plates of Prokudin-Gorskii's self-portrait

Background Context

Almost all 21st-century electronic displays use RGB to display color! In the RGB color model, red, green, and blue primary colors of light are added together to form the full spectrum of human-visible colors. The reason that red, green, and blue were specifically chosen actually has to do with the physiology of the human eye! Humans have 2 kinds of light-sensitive receptor cells in their eyes: cones and rods. Cones are responsible for color vision,

and most humans have 3 kinds of cones: S, M, and L cones. S stands for "short" wavelength, M stands for "medium" wavelength, and L stands for "long" wavelengths. We perceive different wavelengths of light between 400nm and 700nm as different colors. The S cone is most sensitive to light with a wavelength of 440nm (the color blue!). The M cone's light sensitivity peak is 530nm (the color green). And lastly, the L cone's light sensitivity peak is 560nm (the color red). So the RGB model directly corresponds to the 3 colors our eyes are most sensitive to!

Although Sergei Mikhailovich Prokudin-Gorskii (1863-1944) didn't have access to color photography and electronic displays like we do, he had the *foresight* to preserve the Russian Empire of the early 20th century using color. From c. 1905-1915, he took thousands of photographs by using red, green, and blue filters to capture 3 different exposures of every scene. In 1948, the Library of Congress purchased Prokudin-Gorskii's RGB glass plate negatives. From 2000-2004, the LoC scanned Prokudin-Gorskii's glass plates and made them publicly accessible online.

Each glass plate image only captures the intensities of either red, green, or blue light. By stacking the glass plates on top of each other and coloring them, the reds, greens, and blues add together to produce a full color image! However, even though Prokudin-Gorskii's glass plates were exposures of the same scene, the subjects of each plate don't always line up exactly. Naively stacking the plates can result in a final image with blurry edges and colorful ghosts.

In this project, I explored different ways to automatically align the colored glass plates on top of each other in order to minimize visual artifacts.

Overview

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

From a high-level birds-eye-view, my automatic image alignment function follows these steps:

- Takes a digitized image of the blue, green, and red glass plates joined together (sort of like a photostrip), and splits it into 3 equal parts.
- (Bells and whistle) Pre-processes each of the 3 color plates, such as by cropping away the black and white borders on each plate produced as a side-effect of digital scanning.
- Aligns the green plate to the blue plate by searching over a range of possible displacements `(dy, dx)` and finding the displacement that optimizes some metric (e.g. L2 Norm between the pixel values of both plates or normalized-cross-correlation) representing how well the images align together.
- Similarly, aligns the red plate to the blue plate by finding the displacement `(dy, dx)` that results in the best image alignment metric.
- Shifts the green plate by the best displacement found for aligning green to blue. Shifts the red plate by the best displacement found for aligning red to blue. Then stacks all 3 color plates into a single image with 3 color channels.
- (Bells and whistles) Post-process the final color image after stacking, such as by trimming the edges of the final image that don't correspond to pixels where the 3 color plates actually overlapped. For instance, my implementation uses `np.roll` to shift the green and red plates after finding the optimal displacement vectors. `np.roll` wraps pixels from one side to the opposite side, so sometimes certain pixels on the outer edges of the final color image weren't actually in that location in the real scene – they've just been wrapped around to the other side.

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

- Gaze in wonder at the colors of the Russian Empire from over 100 years ago!

The rest of this page will zoom in on the details :)

Single-Scale Alignment

The simplest way to align the color plates is to search for the best alignment at a single scale/resolution of the color plates. I implemented single-scale alignment using a double-nested for-loop that iterated through (dy, dx) pairs. For each candidate displacement, I shifted one plate by the amount (dy, dx) – a positive dy displacement meant shifting the plate DOWN by dy pixels, and a positive dx displacement meant shifting the plate RIGHT by dx pixels. I used this convention to match up with how NumPy sets the origin $(0, 0)$ as corresponding to the top-left corner of an image array, and how the first index in the tuple (i, j) describing the location of the (i, j) -th pixel corresponds to that pixel's row location and the second index corresponds to that pixel's column location.

After shifting one plate, I scored how well this candidate plate aligned with the base plate (in my implementation, I fixed BLUE as the base plate and aligned red and green to the BLUE plate) using the following image matching metrics:

L2 Norm AKA Euclidean Distance: Compute the squared distance between the corresponding pixel values in the candidate image and the base image. Sum the squared distances across all pixel positions. Then take the square root of the sum.

When using the L2 Norm as the alignment metric, a **smaller** L2 norm is better (i.e. the plates are more aligned)!

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

$$L2 = \sqrt{\sum_{i,j} (img1[i,j] - img2[i,j])^2}$$

Normalized Cross-Correlation (NCC): For each image, compute the mean of its pixel values. Center the image matrix by subtracting the mean element-wise from every pixel in the matrix. Then compute the L2 norm of the mean-centered matrix. Normalize by dividing every pixel by this L2 norm. Finally, take the dot product between the two normalized matrices.

When using the NCC as the alignment metric, a **bigger** NCC is better (i.e. the plates are more aligned)!

$$NCC = \left\langle \frac{img1 - \mu_1}{\|img1 - \mu_1\|_2}, \frac{img2 - \mu_2}{\|img2 - \mu_2\|_2} \right\rangle$$

NB: One way to apply these difference metrics to 2D image matrices instead of vectors is by unraveling the matrix into a long, 1-dimensional vector.

Here is a comparison of church.tif, whose best displacement vector differed the most between the two metrics (L2 Norm vs. NCC) among the 14 provided images.

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



church.tif before automatic alignment

church.tif aligned with L2 Norm

red (dy, dx): (58, 188), green (dy, dx): (25, 2)

church.tif aligned with NCC

red (dy, dx): (57, -5), green (dy, dx): (24, 3)

Single-Scale Gallery

Using single-scale automatic alignment and searching over a range of $[-15, 15]$ for dy and dx worked well on the smaller digitized glass plate data! Here are the results of single-scale automatic alignment on the smaller images: cathedral.jpg, monastery.jpg, and tobolsk.jpg.

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



cathedral.jpg (original,
dimensions: (341px, 390px))



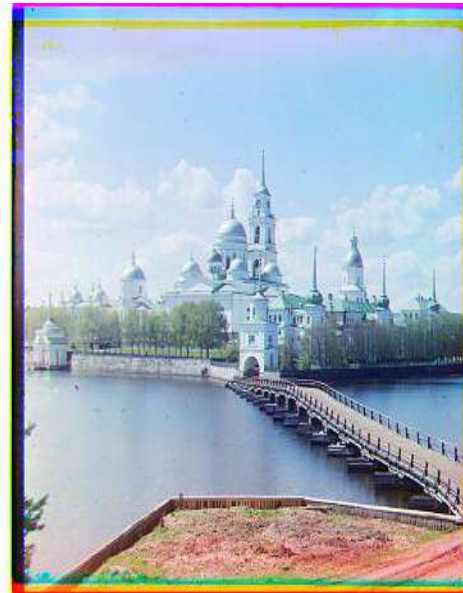
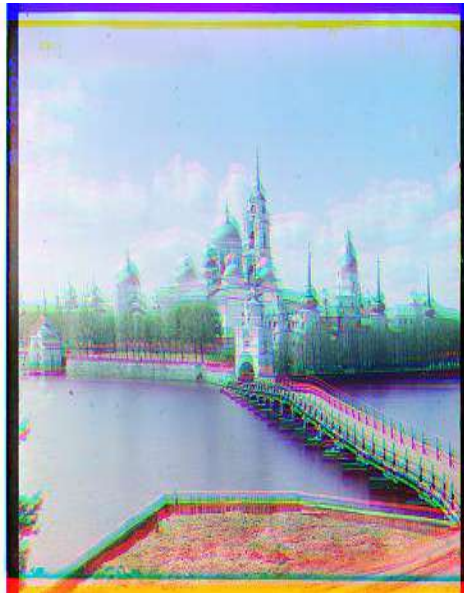
cathedral.jpg (single-scale
+ L2)

red (dy, dx): (12, 3), green (dy,
dx): (5, 2); total time: 0.218s



cathedral.jpg (single-scale
+ NCC)

red (dy, dx): (12, 3), green (dy,
dx): (5, 2); total time: 0.535s



Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

monastery.jpg (original,
dimensions: (341px, 391px))



monastery.jpg (single-scale
+ L2)

red (dy, dx): (3, 2), green (dy,
dx): (-3, 2); total time: 0.234s

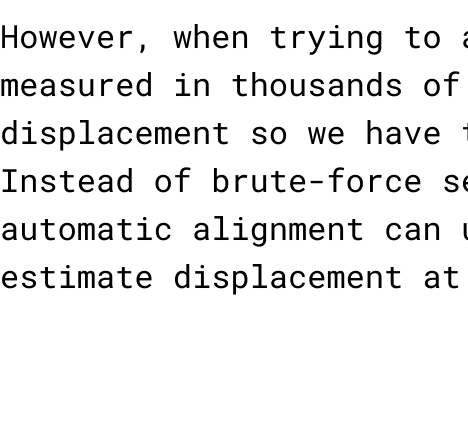


monastery.jpg (single-scale
+ NCC)

red (dy, dx): (3, 2), green (dy,
dx): (-3, 2); total time: 0.504s

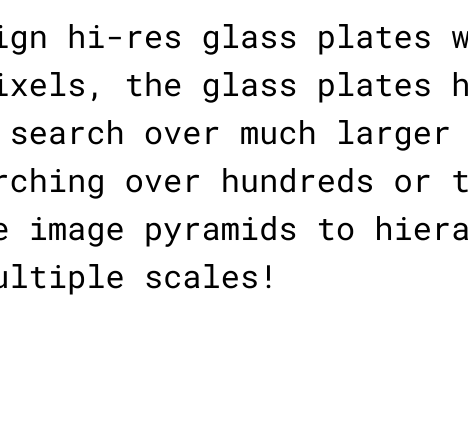


tobolsk.jpg (original,
dimensions: (341px, 396px))



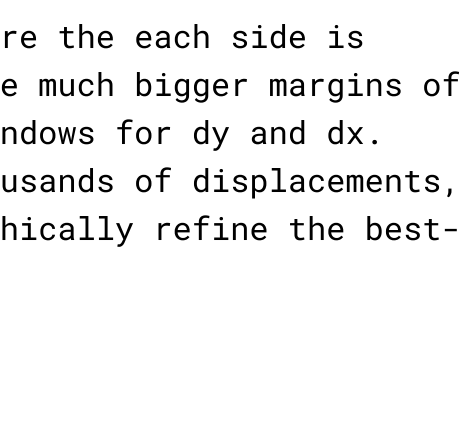
tobolsk.jpg (single-scale +
L2)

red (dy, dx): (6, 3), green (dy,
dx): (3, 2); total time: 0.225s



tobolsk.jpg (single-scale +
NCC)

red (dy, dx): (6, 3), green (dy,
dx): (3, 2); total time: 0.525s



Contents

Background
Context

Overview

Single-Scale
Alignment

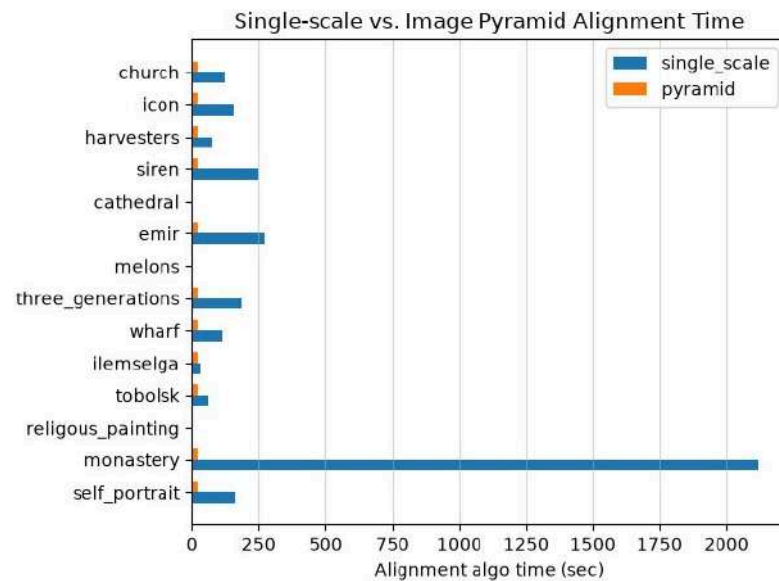
Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

However, when trying to align hi-res glass plates where the each side is measured in thousands of pixels, the glass plates have much bigger margins of displacement so we have to search over much larger windows for dy and dx. Instead of brute-force searching over hundreds or thousands of displacements, automatic alignment can use image pyramids to hierarchically refine the best-estimate displacement at multiple scales!



The time in seconds it took the single-scale alignment algo to search the same window of possible displacements as the best displacements found by the image pyramid implementation

Image Pyramids

Contents

Background
Context

Overview

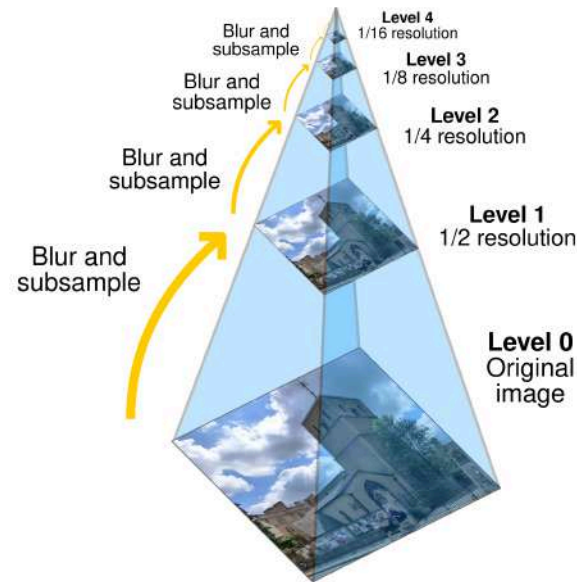
Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



An image pyramid with 5 levels/resolution scales (source: Wikipedia)

An image pyramid is a strategy in image-processing algorithms where the image is scaled to different resolutions and processed hierarchically. I incorporated image pyramids into my automatic alignment algorithm by recursively calling the automatic alignment algorithm on a downsampled version of the color plate and the base plate, where each time I rescaled the dimensions by $1/2$. I set the base case to be when the plates shrunk down to a width or height of under 200 px. At the base case, I simply performed a brute-force search over a window of $[-15, 15]$ with $(0, 0)$ as the initial estimate for the displacements (dy, dx) . The base case returns a displacement vector (dy, dx) that represents the current best estimate for how to align the two plates at this coarse resolution.

Back in the recursive caller, I first scale the returned displacement vector (dy, dx) by $2x$ – the raw (dy, dx) represents how much the downsampled plates with $1/2$ the width and $1/2$ the height should be displaced to align with each other. Thus, these displacements should be scaled back up by a factor of $2x$ to

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

correspond to the actual number of pixels that the higher resolution plates would need to be shifted by.

The algorithm uses these scaled up displacement vectors as the centers for a new search neighborhood, this time with a smaller, more manageable search radius for dy and dx . If the algorithm finds a displacement with a better alignment metric during the course of searching in this new neighborhood, it saves the updated displacements. Finally, the algorithm returns the best displacements it's found so far after refining the initial estimates.

This continues recursively back up the call stack until control flow returns back to the first automatic alignment function call, with the plates at their original, full dimensions.

Of course, an image is worth a thousand coding experiments! Here are some additional features I implemented in my image pyramid automatic alignment algorithm to #makeitwork.

- Ignoring the margins of the plates in the displacement search sub-

function: I observed that for many of the images, all 3 color plates had noticeable black or white borders. Since the pixel values are mostly the same for all 3 color plates along these borders, it's likely that the similarity of these pixels on the border would dominate the alignment metrics even though they don't provide useful signal for actually aligning the most important visual subjects in the scene. Therefore, in my displacement search sub-function, I included an adjustable margin to crop off from all 4 sides of the two plates before searching to remove the influence of these low-signal borders on the metric.

- Scoring the interior region of the two plates rather than the full image:

I hypothesized that most of the most useful signal for aligning the two plates together would be in the center of the image, away from the edges

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

which could contain extraneous artifacts as a result of preservation or scanning. At the same time, scoring the largest interior rectangle possible would allow my algorithm to compare the largest proportion of pixels between the two plates for matches.

So before searching, I first computed the largest possible interior rectangle from the base plate.

I knew that given initial displacements `(dy_center, dx_center)` and radii `(dy_radius, dx_radius)`, the algorithm would shift the other plate by some vertical displacement within the range `[dy_center - dy_radius, dy_center + dy_radius]` and by some horizontal displacement within the range `[dx_center - dx_radius, dx_center + dx_radius]`.

Based on this heuristic, I calculated left edges, right edges, top edges, and bottom edges of the base plate's interior rectangle such that even if the algo shifted the other plate, the regions of the shifted plate that would now be located on top of the chosen rectangle from the base plate would still be within bounds of the shifted plate; and therefore I could slice out a corresponding rectangle from the shifted plate to compare with the base plate's interior rectangle without going out of bounds.

For instance, assume that the rectangle from the base plate has the following coordinates: `y0` (top edge), `y1` (bottom edge), `x0` (left edge), and `x1` (right edge). This rectangle can be extracted from the base plate as follows: `base_plate[y0:y1, x0:x1]`. Also assume that the other color plate is being shifted by `(dy, dx)`, where positive `dy` is a shift downwards and positive `dx` is a shift rightwards. Then the pixels from the other color plate that are now mapped on top of the base plate's chosen rectangle are `plate[y0 - dy : y1 - dy, x0 - dx : x1 - dx]`. Thus, I needed to make sure

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

that this change-of-coordinates always stayed within bounds of the other color plate, and also didn't index into the ignored margins!

- **Trimming non-overlapping pixels from the edges of the final color image after all plates were stacked:** When using `np.roll` to shift the pixels of the G and R plates, sometimes the pixels on one edge are wrapped around to the opposite edge. As a result, the edges of the final color image might contain some pixels which were actually on the opposite side of the real scene. I use the displacement vectors for the G plates and R plates to calculate the smallest margin I can trim away on each of the 4 sides of the final stacked image to remove these non-sequitur pixels.

Bells and Whistles!

Automatic cropping and detecting solid-color borders: Before aligning the color plates to one another, I cropped the white and black borders from all 3 plates. To detect whether a row or column should be cropped, I computed the fraction of elements in a given row/column whose pixel values were within a tolerance of 0.1 from 1.0 (white) or 0.1 (an approximation for black). As long as more than 70% of the pixels in this row/column were the target color, the algorithm considered them part of the solid-color border. The algorithm would then continue scanning inwards until it found a row/column where less than 70% of the pixels were within tolerance of the solid-color border, or until it hit a max crop limit of 5% of the full height/width.

Automatic edge detection using Canny edge detectors: Even with all the automatic alignment tricks I threw at it so far, `emir.tif` evaded alignment! Aligning based on pixel intensity alone proved insufficient because Emir's robe is overwhelmingly blue. The pixel values of his robe in the red and green plates are much darker compared to to the blue plate; hence, L2 norm and NCC

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

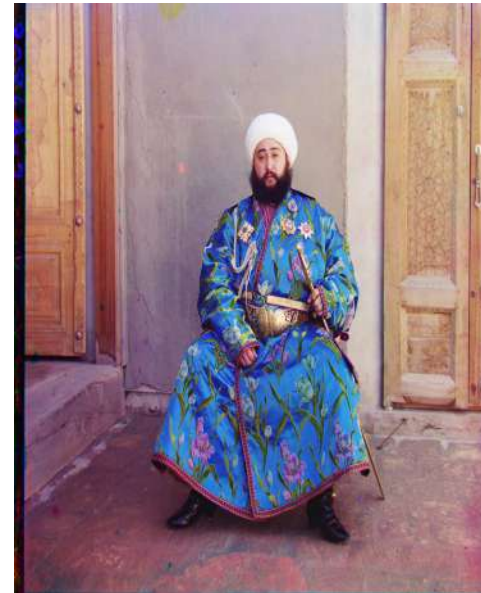
Final
Results
Gallery

will struggle to align his robe's red or green pixels to the blue pixels since their values differ so much! The solution I explore was to align the color plates based on edges of objects, people, or scenery instead! Since the three color plates all captured the same scene at roughly the same moment in time, the shapes in the scene were more consistent than color, especially if one color dominates the other two in a particular spot. I used scikit-image's Canny edge detector. Canny edge detection is a multi-stage algorithm that detects edges by convolving the image with a Gaussian filters to remove noise and taking the gradient of the image to compute how the image's pixel intensities change in a local window. The intensity of the image gradient can be used to identify edges.

Below, you can see how edge detection greatly improved the final alignment quality of Emir's portrait! For the other provided images, automatic alignment based on pixel intensities vs. edges produced roughly the same displacements.



emir.tif (original)



Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

```
emir.tif (image pyramid +      emir.tif (image pyramid +  
      NCC)                      Canny + NCC)  
red (dy, dx): (132, -276), green  red (dy, dx): (107, 40), green  
      (dy, dx): (49, 24)          (dy, dx): (49, 23)
```

Final Results Gallery

Here are the final results of my automatic alignment algorithm on 14 provided glass plate images, plus 4 pictures I chose from the Library of Congress's Prokudin-Gorskii online collection: [laika.tif](#), [milan_duomo.tif](#), [borodino.tif](#), and [st_boris.tif](#)!

These were all processed by first splitting the image data into red, green, and blue plates; cropping solid-color borders from each of the glass plates; aligning the red plate to the blue plate using my image pyramid alignment algorithm; aligning the green plate to the blue plate using my image pyramid alignment algorithm; stacking the 3 aligned plates together into a 3-channel color image; and lastly, trimming the misplaced, wrap-around pixels from the edges.

Overall, I found cropping solid-color borders and using NCC produced higher quality alignments. Edge detection most noticeably improved the quality of [emir.tif](#), but found pretty similar best displacements for most of the other images.

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



cathedral.jpg
(original)



cathedral.jpg (image
pyramid + L2)

red (dy, dx): (12, 3),
green (dy, dx): (5, 2)



cathedral.jpg (image
pyramid + NCC)

red (dy, dx): (12, 3),
green (dy, dx): (5, 2)

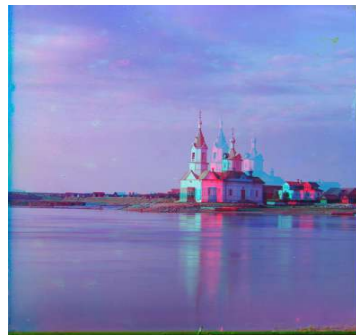


cathedral.jpg (image
pyramid + Canny +
NCC)

red (dy, dx): (12, 3),
green (dy, dx): (5, 2)



church.tif
(original)



church.tif (image
pyramid + L2)

red (dy, dx): (58, 188),
green (dy, dx): (25, 2)



church.tif (image
pyramid + NCC)

red (dy, dx): (57, -5),
green (dy, dx): (24, 3)



church.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (58, -4),
green (dy, dx): (25, 4)

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



emir.tif (original)

emir.tif (image
pyramid + L2)

red (dy, dx): (104, 43),
green (dy, dx): (49, 24)

emir.tif (image
pyramid + NCC)

red (dy, dx): (132,
-276), green (dy, dx):
(49, 24)

emir.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (107, 40),
green (dy, dx): (49, 23)

harvesters.tif
(original)harvesters.tif
(image pyramid + L2)

red (dy, dx): (124, 14),
green (dy, dx): (60, 16)

harvesters.tif
(image pyramid +
NCC)

red (dy, dx): (124, 13),
green (dy, dx): (60, 16)

harvesters.tif
(image pyramid +
Canny + NCC)

red (dy, dx): (123, 9),
green (dy, dx): (60, 18)

Contents

Background
Context

Overview

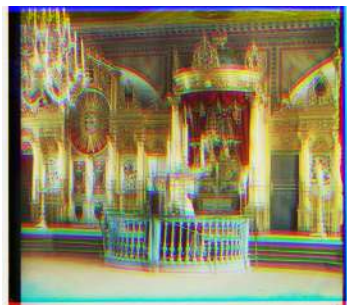
Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



icon.tif (original)



icon.tif (image
pyramid + L2)

red (dy, dx): (90, 23),
green (dy, dx): (41, 17)



icon.tif (image
pyramid + NCC)

red (dy, dx): (89, 23),
green (dy, dx): (40, 16)



icon.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (88, 22),
green (dy, dx): (38, 16)



ilemselga.tif
(original)



ilemselga.tif (image
pyramid + L2)

red (dy, dx): (129, 11),
green (dy, dx): (39, 7)



ilemselga.tif (image
pyramid + NCC)

red (dy, dx): (130, 11),
green (dy, dx): (39, 7)



ilemselga.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (130, 11),
green (dy, dx): (39, 7)

Contents

Background
Context

Overview

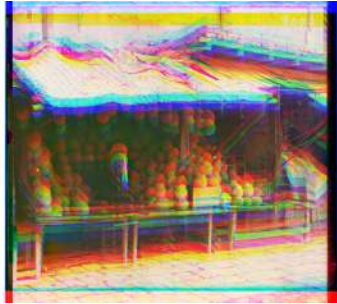
Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



melons.tif
(original)



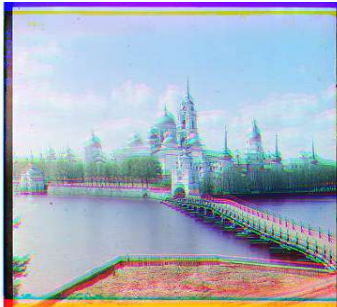
melons.tif (image
pyramid + L2)
red (dy, dx): (179, 13),
green (dy, dx): (82, 10)



melons.tif (image
pyramid + NCC)
red (dy, dx): (178, 12),
green (dy, dx): (82, 9)



melons.tif (image
pyramid + Canny +
NCC)
red (dy, dx): (182, 11),
green (dy, dx): (79, 9)



monastery.jpg
(original)



monastery.jpg (image
pyramid + L2)
red (dy, dx): (3, 2),
green (dy, dx): (-3, 2)



monastery.jpg (image
pyramid + NCC)
red (dy, dx): (3, 2),
green (dy, dx): (-3, 2)



monastery.jpg (image
pyramid + Canny +
NCC)
red (dy, dx): (3, 2),
green (dy, dx): (-3, 2)

Contents

Background
Context

Overview

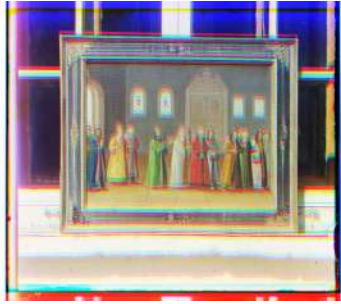
Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



religious_painting.tif
(original)



religious_painting.tif
(image pyramid + L2)

red (dy, dx): (68, 7),
green (dy, dx): (28, 5)



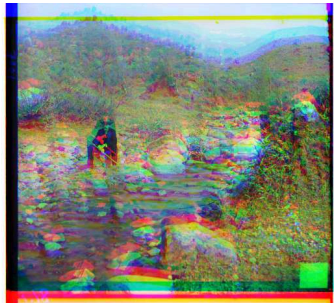
religious_painting.tif
(image pyramid + NCC)

red (dy, dx): (68, 7),
green (dy, dx): (28, 5)



religious_painting.tif
(image pyramid + Canny
+ NCC)

red (dy, dx): (69, 6),
green (dy, dx): (29, 1)



self_portrait.tif
(original)



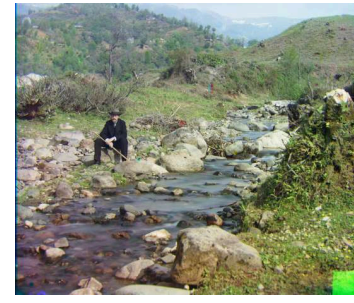
self_portrait.tif
(image pyramid + L2)

red (dy, dx): (176, 36),
green (dy, dx): (79, 28)



self_portrait.tif
(image pyramid +
NCC)

red (dy, dx): (176, 36),
green (dy, dx): (79, 28)



self_portrait.tif
(image pyramid +
Canny + NCC)

red (dy, dx): (175, 37),
green (dy, dx): (77, 29)

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



siren.tif (original)

siren.tif (image
pyramid + L2)

red (dy, dx): (96, -25),
green (dy, dx): (49, -6)

siren.tif (image
pyramid + NCC)

red (dy, dx): (96, -25),
green (dy, dx): (49, -6)

siren.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (96, -23),
green (dy, dx): (48, -8)

three_generations.tif
(original)three_generations.tif
(image pyramid + L2)

red (dy, dx): (112, 10),
green (dy, dx): (54, 12)

three_generations.tif
(image pyramid + NCC)

red (dy, dx): (112, 10),
green (dy, dx): (54, 12)

three_generations.tif
(image pyramid +
Canny + NCC)

red (dy, dx): (111, 8),
green (dy, dx): (56, 12)

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery



tobolsk.jpg
(original)



tobolsk.jpg (image
pyramid + L2)

red (dy, dx): (6, 3),
green (dy, dx): (3, 3)



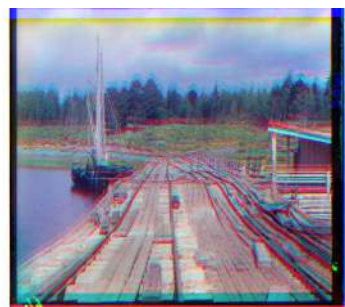
tobolsk.jpg (image
pyramid + NCC)

red (dy, dx): (6, 3),
green (dy, dx): (3, 3)



tobolsk.jpg (image
pyramid + Canny +
NCC)

red (dy, dx): (6, 3),
green (dy, dx): (3, 3)



wharf.tif (original)



wharf.tif (image
pyramid + L2)

red (dy, dx): (83, -17),
green (dy, dx): (15, -7)



wharf.tif (image
pyramid + NCC)

red (dy, dx): (83, -17),
green (dy, dx): (15, -7)



wharf.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (83, -16),
green (dy, dx): (15, -7)

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

Additional Images



laika.tif (original)

laika.tif (image
pyramid + L2)

red (dy, dx): (115, 7),
green (dy, dx): (27, 3)

laika.tif (image
pyramid + NCC)

red (dy, dx): (115, 7),
green (dy, dx): (27, 2)

laika.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (116, 4),
green (dy, dx): (28, 1)

milan_duomo.tif
(original)milan_duomo.tif
(image pyramid + L2)

red (dy, dx): (111,
-15), green (dy, dx):
(47, -9)

milan_duomo.tif
(image pyramid +
NCC)

red (dy, dx): (111,
-15), green (dy, dx):
(47, -9)

milan_duomo.tif
(image pyramid +
Canny + NCC)

red (dy, dx): (110,
-17), green (dy, dx):
(50, -6)



Contents

Background
Context

Overview

Single-Scale
Alignment

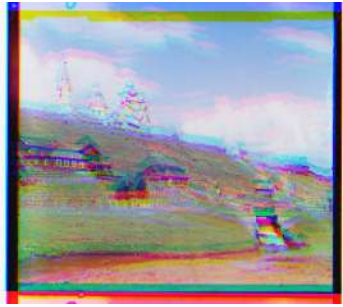
Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery

borodino.tif
(original)



st_boris.tif
(original)

borodino.tif (image
pyramid + L2)

red (dy, dx): (109, 39),
green (dy, dx): (53, 28)



st_boris.tif (image
pyramid + L2)

red (dy, dx): (158, 62),
green (dy, dx): (65, 33)

borodino.tif (image
pyramid + NCC)

red (dy, dx): (109, 40),
green (dy, dx): (54, 29)



st_boris.tif (image
pyramid + NCC)

red (dy, dx): (158, 62),
green (dy, dx): (65, 33)

borodino.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (110, 42),
green (dy, dx): (51, 24)



st_boris.tif (image
pyramid + Canny +
NCC)

red (dy, dx): (157, 61),
green (dy, dx): (65, 35)

Contents

Background
Context

Overview

Single-Scale
Alignment

Single-Scale
Gallery

Image
Pyramids

Bells and
Whistles

Final
Results
Gallery